

Title:	Dependent Execution for a Unified Executors Proposal for C++
Authors:	Jared Hoberock, jhoberock@nvidia.com Michael Garland, mgarland@nvidia.com Chris Kohlhoff, chris@kohlhoff.com Chris Mysen, mysen@google.com Carter Edwards, hcedwar@sandia.gov Gordon Brown, gordon@codeplay.com
Other Contributors:	Hans Boehm, hboehm@google.com Thomas Heller, thom.heller@gmail.com Lee Howes, lwh@fb.com Bryce Leibelbach, bryceleibach@gmail.com Hartmut Kaiser, hartmut.kaiser@gmail.com Bryce Leibelbach, bryceleibach@gmail.com Gor Nishanov, gorn@microsoft.com Thomas Rodgers, rodgert@twrogers.com David Hollman, dshollm@sandia.gov Michael Wong, michael@codeplay.com
Document Number:	P1244R0
Date:	2018-10-08
Audience:	SG1 - Concurrency and Parallelism, LEWG
Reply-to:	sg1-exec@googlegroups.com
Abstract:	This paper extends P0443 's executors programming model with functionality to support dependent execution.

0.1 Changelog

0.1.1 Revision 0

Initial revision.

As directed by the SG1/LEWG straw poll taken during the 2018 Bellevue executors meeting, we have separated The Unified Executors programming model proposal into two papers. [P0443](#) contains material related to one-way execution which the authors hope to standardize with C++20 as suggested by the Bellevue poll. This document contains remaining material related to dependent execution. We expect the form of this paper's material to evolve as committee consensus builds around a design for dependent execution.

1 Proposed Wording

1.0.1 Header <exception> synopsis

```
namespace std {
namespace experimental {
inline namespace executors_v1 {

    // Exception argument tag
    struct exception_arg_t { explicit exception_arg_t() = default; };
    inline constexpr exception_arg_t exception_arg{};

}
}
}
```

1.1 Exception argument tag

The `exception_arg_t` struct is an empty structure type used as a unique type to disambiguate constructor and function overloading. Specifically, functions passed to `then_execute` and `bulk_then_execute` may have `exception_arg_t` as an argument, immediately followed by an argument that should be interpreted as an exception thrown from a preceding function invocation.

1.2 Execution Support Library

1.2.1 Header <execution> synopsis

```
namespace std {
namespace experimental {
inline namespace executors_v1 {
namespace execution {
```

```

// Interface-changing properties:

struct twoway_t;
struct then_t;
struct bulk_twoway_t;
struct bulk_then_t;

constexpr twoway_t twoway;
constexpr then_t then;
constexpr bulk_twoway_t bulk_twoway;
constexpr bulk_then_t bulk_then;

// Executor type traits:

template<class Executor> struct is_twoway_executor;
template<class Executor> struct is_then_executor;
template<class Executor> struct is_bulk_twoway_executor;
template<class Executor> struct is_bulk_then_executor;

template<class Executor> constexpr bool is_twoway_executor_v = is_twoway_executor<Executor>::value;
template<class Executor> constexpr bool is_then_executor_v = is_then_executor<Executor>::value;
template<class Executor> constexpr bool is_bulk_twoway_executor_v = is_bulk_twoway_executor<Executor>::value;
template<class Executor> constexpr bool is_bulk_then_executor_v = is_bulk_then_executor<Executor>::value;

template<class Executor, class T> struct executor_future;

template<class Executor, class T> using executor_future_t = typename executor_future<Executor, T>::type;

} // namespace execution
} // inline namespace executors_v1
} // namespace experimental
} // namespace std

```

1.3 Requirements

1.3.1 Future requirements

A type `F` meets the `Future` requirements for some value type `T` if `F` is `std::experimental::future<T>` (defined in the C++ Concurrency TS, ISO/IEC TS 19571:2016). [Note: This concept is included as a placeholder to be elaborated, with the expectation that the elaborated requirements for `Future` will expand the applicability of some executor customization points. --end note]

Forward progress guarantees are a property of the concrete `Future` type. [Note: `std::experimental::future<T>::wait()` blocks with forward progress guarantee delegation until the shared state is ready. --end note]

1.3.2 TwoWayExecutor requirements

The `TwoWayExecutor` requirements specify requirements for executors which submit function objects with a channel for awaiting the completion of a submitted function object and obtaining its result.

A type `X` satisfies the `TwoWayExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the Table below.

In the Table below,

- `x` denotes a (possibly const) executor object of type `X`,
- `cf` denotes the function object `DECAY_COPY(std::forward<F>(f))`
- `f` denotes a function object of type `F&&` invocable as `cf()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements,
- `R` denotes the type of the expression `cf()`.

Expression	Return Type	Operational semantics
<code>x.twoway_execute(f)</code>	A type that satisfies the <code>Future</code> requirements for the value type <code>R</code> .	Evaluates <code>DECAY_COPY(std::forward<F>(f))</code> on the calling thread to create <code>cf</code> that will be invoked at most once by an execution agent. May block pending completion of this function object. Synchronizes with [intro.multithread] the invocation of <code>f</code>. Stores the result of the invocation, or any exception thrown by the invocation, in the associated shared state of the returned <code>Future</code>.

1.3.3 ThenExecutor requirements

The `ThenExecutor` requirements specify requirements for executors which submit function objects whose invocation is predicated on the readiness of a specified future, and which provide a channel for awaiting the completion of the submitted function object and obtaining its result.

A type `X` satisfies the `ThenExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the Table below.

In the Table below,

- `x` denotes a (possibly `const`) executor object of type `X`,
- `fut` denotes a future object satisfying the `Future` requirements,
- `val` denotes any object stored in `fut`'s associated shared state when it becomes nonexceptionally ready,
- `e` denotes the object stored in `fut`'s associated shared state when it becomes exceptionally ready,
- `cf` denotes the function object `DECAY_COPY(std::forward<F>(f))`,
- `NORMAL` denotes the expression `cf(val)` if `fut`'s value type is non-void and `cf()` if `fut`'s value type is void,
- `EXCEPTIONAL` denotes the expression `cf(exception_arg, e)`,
- `f` denotes a function object of type `F&&` invocable as `NORMAL` or `EXCEPTIONAL` and where `decay_t<F>` satisfies the `MoveConstructible` requirements,
- and `R` denotes the type of the expression `NORMAL`.

Expression	Return Type	Operational semantics
<code>x.then_execute(f, fut)</code>	A type that satisfies the <code>Future</code> requirements for the value type <code>R</code> .	Evaluates <code>DECAY_COPY(std::forward<F>(f))</code> on the calling thread to create <code>cf</code>. When <code>fut</code> becomes nonexceptionally ready and if <code>NORMAL</code> is a well-formed expression then <code>NORMAL</code> is invoked by an execution agent at most once. Otherwise, when <code>fut</code> becomes exceptionally ready and if <code>EXCEPTIONAL</code> is a well-formed expression then <code>EXCEPTIONAL</code> is invoked at most once by an execution agent. If <code>NORMAL</code> and <code>EXCEPTIONAL</code> are both well-formed expressions, <code>decltype(EXCEPTIONAL)</code> shall be convertible to <code>R</code>. If <code>NORMAL</code> is not a well-formed expression and <code>EXCEPTIONAL</code> is a well-formed expression, <code>decltype(EXCEPTIONAL)</code> shall be convertible to <code>decltype(val)</code>. If neither <code>NORMAL</code> nor <code>EXCEPTIONAL</code> are well-formed expressions, the invocation of <code>then_execute</code> is ill-formed. May block pending completion of <code>NORMAL</code> or <code>EXCEPTIONAL</code>. Synchronizes with <code>[intro.multithread]</code> the invocation of <code>f</code>. Stores the result of either the <code>NORMAL</code> or <code>EXCEPTIONAL</code> expression, or any exception thrown by either, in the associated shared state of the returned <code>Future</code>. Otherwise, stores either <code>val</code> or <code>e</code> in the associated shared state of the returned <code>Future</code>.

1.3.4 BulkTwoWayExecutor requirements

The `BulkTwoWayExecutor` requirements specify requirements for executors which submit a function object with a channel for awaiting the completion of the submitted function object and obtaining its result.

A type `X` satisfies the `BulkTwoWayExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the Table below.

In the Table below,

- `x` denotes a (possibly `const`) executor object of type `X`,
- `n` denotes a shape object whose type is `executor_shape_t<X>`,
- `rf` denotes a `CopyConstructible` function object with zero arguments whose result type is `R`,
- `sf` denotes a `CopyConstructible` function object with zero arguments whose result type is `S`,
- `i` denotes a (possibly `const`) object whose type is `executor_index_t<X>`,
- `s` denotes an object whose type is `S`,
- `cf` denotes the function object `DECAY_COPY(std::forward<F>(f))`,
- if `R` is non-void,
 - `r` denotes an object whose type is `R`,
 - `INVOKE_CF` denotes the expression `cf(i, r, s)`,

- if R is void,
 - `INVOKE_CF` denotes the expression `cf(i, r, s)`,
- `f` denotes a function object of type `F&&` invocable as `INVOKE_CF` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
<code>x.bulk_twoway_execute(f, n, rf, sf)</code>	A type that satisfies the Future requirements for the value type R.	Evaluates <code>DECAY_COPY(std::forward<F>(f))</code> on the calling thread to create a function object <code>cf</code>. <i>[Note: * Additional copies of <code>cf</code> may subsequently be created. *--end note]</i> For each value of <code>i</code> in shape <code>n</code>, <code>INVOKE_CF</code> (possibly with copy of <code>cf</code>) will be invoked at most once by an execution agent that is unique for each value of <code>i</code>. If R is non-void, <code>rf()</code> will be invoked at most once to produce the value <code>r</code>. <code>sf()</code> will be invoked at most once to produce the value <code>s</code>. May block pending completion of one or more invocations of <code>cf</code>. Synchronizes with <code>[intro.multithread]</code> the invocations of <code>f</code>. Once all invocations of <code>f</code> finish execution, <code>r</code> or any exception thrown by an invocation of <code>f</code> are stored in the associated shared state of the returned Future.

1.3.5 BulkThenExecutor requirements

The `BulkThenExecutor` requirements specify requirements for executors which submit function objects whose initiation is predicated on the readiness of a specified future, and which provide a channel for awaiting the completion of the submitted function object and obtaining its result.

A type `X` satisfies the `BulkThenExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the Table below.

In the Table below,

- `x` denotes a (possibly const) executor object of type `X`,
- `n` denotes a shape object whose type is `executor_shape_t<X>`,
- `fut` denotes a future object satisfying the Future requirements,
- `val` denotes any object stored in `fut`'s associated shared state when it becomes nonexceptionally ready,
- `e` denotes the object stored in `fut`'s associated shared state when it becomes exceptionally ready,
- `rf` denotes a `CopyConstructible` function object with zero arguments whose result type is R,
- `sf` denotes a `CopyConstructible` function object with zero arguments whose result type is S,
- `i` denotes a (possibly const) object whose type is `executor_index_t<X>`,
- `s` denotes an object whose type is S,
- `cf` denotes the function object `DECAY_COPY(std::forward<F>(f))`,
- if R is non-void,
 - `r` denotes an object whose type is R,
 - `NORMAL` denotes the expression `cf(i, val, r, s)`,
 - `EXCEPTIONAL` denotes the expression `cf(exception_arg, e, r, s)`,
- if R is void,
 - `NORMAL` denotes the expression `cf(i, val, s)`,
 - `EXCEPTIONAL` denotes the expression `cf(exception_arg, e, s)`,
- `f` denotes a function object of type `F&&` invocable as `NORMAL` or `EXCEPTIONAL` and where `decay_t<F>` satisfies the `MoveConstructible` requirements,

Expression	Return Type	Operational semantics
<code>x.bulk_then_execute(f, n, fut, rf, sf)</code>	A type that satisfies the Future requirements for the value type R.	Evaluates <code>DECAY_COPY(std::forward<F>(f))</code> on the calling thread to create function object <code>cf</code>. <i>[Note: * Additional copies of <code>cf</code> may subsequently be created. *--end note]</i> When <code>fut</code> becomes nonexceptionally ready and if <code>NORMAL</code> is a well-formed expression then for each value of <code>i</code> in shape <code>n</code>, <code>NORMAL</code> (possibly with copy of <code>cf</code>) will be invoked at most once by an execution agent that is unique for each value of <code>i</code>. If R is non-void then <code>rf()</code> will be invoked at most once to produce the value <code>r</code>. <code>sf()</code> will be invoked at most once to produce the value <code>s</code>.

Expression

Return Type

Operational semantics

Otherwise, when `fut` becomes exceptionally ready and if `EXCEPTIONAL` is a well-formed expression then `EXCEPTIONAL` is invoked at most once by an execution agent.

If `NORMAL` and `EXCEPTIONAL` are both well-formed expressions, `decltype(EXCEPTIONAL)` shall be convertible to `R`.

If `NORMAL` is not a well-formed expression and `EXCEPTIONAL` is a well-formed expression, `decltype(EXCEPTIONAL)` shall be convertible to `decltype(val)`.

If neither `NORMAL` nor `EXCEPTIONAL` are well-formed expressions, the invocation of `then_execute` is ill-formed.

May block pending completion of `NORMAL` or `EXCEPTIONAL`.

Synchronizes with `[intro.multithread]` the invocation of `f`.

Stores the result of either the `NORMAL` or `EXCEPTIONAL` expression, or any exception thrown by either, in the associated shared state of the returned `Future`.

Otherwise, stores either `val` or `e` in the associated shared state of the returned `Future`.

1.4 Executor properties

1.4.1 Interface-changing properties

```
struct twoway_t;  
struct then_t;  
struct bulk_twoway_t;  
struct bulk_then_t;
```

Property	Requirements
<code>twoway_t</code>	The executor type satisfies the <code>TwoWayExecutor</code> requirements.
<code>then_t</code>	The executor type satisfies the <code>ThenExecutor</code> requirements.
<code>bulk_twoway_t</code>	The executor type satisfies the <code>BulkTwoWayExecutor</code> requirements.
<code>bulk_then_t</code>	The executor type satisfies the <code>BulkThenExecutor</code> requirements.

The `std::execution::oneway_t`, `twoway_t`, `then_t`, `std::execution::bulk_oneway_t`, `bulk_twoway_t` and `bulk_then_t` properties are mutually exclusive.

1.4.1.1 `twoway_t` customization points

In addition to conforming to the above specification, the `twoway_t` property provides the following customization:

```
struct twoway_t  
{  
    template<class Executor>  
        friend see-below require(Executor ex, twoway_t);  
};
```

This customization point returns an executor that satisfies the `twoway_t` requirements by adapting the native functionality of an executor that does not satisfy the `twoway_t` requirements.

```
template<class Executor>  
    friend see-below require(Executor ex, twoway_t);
```

Returns: A value `e1` of type `E1` that holds a copy of `ex`. `E1` has member functions `require` and `query` that forward to the corresponding members of the copy of `ex`, if present. For some type `T`, the type yielded by `executor_future_t<E1, T>` is `executor_future_t<Executor, T>` if `then_t::static_query_v<Executor>` is true; otherwise, it is `std::experimental::future<T>`. `e1` has the same properties as `ex`, except for the addition of the `twoway_t` property and the exclusion of other interface-changing properties. The type `E1` satisfies the `TwoWayExecutor` requirements as follows:

- If `bulk_twoway_t::static_query_v<Executor>` is true, then E1 implements member function `twoway_execute` in terms of the member function `bulk_twoway_execute` of the object `ex`, and E1 has a member function `Executor require(bulk_twoway_t) const` that returns a copy of `ex`.
- If `then_t::static_query_v<Executor>` is true, then E1 implements member function `twoway_execute` in terms of the member function `then_execute` of the object `ex`, and E1 has a member function `Executor require(then_t) const` that returns a copy of `ex`.
- If `bulk_then_t::static_query_v<Executor>` is true, then E1 implements member function `twoway_execute` in terms of the member function `bulk_then_execute` of the object `ex`, and E1 has a member function `Executor require(bulk_then_t) const` that returns a copy of `ex`.
- If `std::execution::oneway_t::static_query_v<Executor>` && `adaptable_blocking_t::static_query_v<Executor>` is true, then E1 implements member function `twoway_execute` in terms of the member function `execute` of the object `ex`, and E1 has a member function `Executor require(std::execution::oneway_t) const` that returns a copy of `ex`.
- If `std::execution::bulk_oneway_t::static_query_v<Executor>` && `adaptable_blocking_t::static_query_v<Executor>` is true, then E1 implements member function `twoway_execute` in terms of the member function `bulk_execute` of the object `ex`, and E1 has a member function `Executor require(std::execution::bulk_oneway_t) const` that returns a copy of `ex`.

Remarks: This function shall not participate in overload resolution unless `twoway_t::template static_query_v<Executor>` is false and `bulk_twoway_t::static_query_v<Executor>` || `then_t::static_query_v<Executor>` || `bulk_then_t::static_query_v<Executor>` || `(std::execution::oneway_t::static_query_v<Executor> && adaptable_blocking_t::static_query_v<Executor>)` || `(std::execution::bulk_oneway_t::static_query_v<Executor> && adaptable_blocking_t::static_query_v<Executor>)` is true.

1.4.1.2 twoway_t polymorphic wrapper

In addition to conforming to the specification for polymorphic executor wrappers, the nested class template `twoway_t::polymorphic_executor_type` provides the following member functions:

```
template <class... SupportableProperties>
class polymorphic_executor_type
{
public:
    template<class Executor>
        polymorphic_executor_type(Executor e);

    template<class Executor>
        polymorphic_executor_type& operator=(Executor e);

    template<class Function>
        std::experimental::future<result_of_t<decay_t<Function>()>>
            twoway_execute(Function&& f) const
};
```

`twoway_t::polymorphic_executor_type` satisfies the `TwoWayExecutor` requirements.

```
template<class Executor>
    polymorphic_executor_type(Executor e);
```

Remarks: This function shall not participate in overload resolution unless:

- `can_require_v<Executor, twoway_t>`.
- `can_require_v<Executor, P>`, if `P::is_requirable`, where `P` is each property in `SupportableProperties...`.
- `can_prefer_v<Executor, P>`, if `P::is_preferable`, where `P` is each property in `SupportableProperties...`.
- and `can_query_v<Executor, P>`, if `P::is_requirable == false` and `P::is_preferable == false`, where `P` is each property in `SupportableProperties...`.

Effects: *this targets a copy of `e1`, where `e1` is the result of `execution::require(e, twoway)`.

```
template<class Executor>
    polymorphic_executor_type& operator=(Executor e);
```

Requires: As for `template<class Executor> polymorphic_executor_type(Executor e)`.

Effects: `polymorphic_executor_type(std::move(e)).swap(*this)`.

Returns: *this.

```
template<class Function>
    std::experimental::future<result_of_t<decay_t<Function>()>>
        twoway_execute(Function&& f) const
```

Remarks: This function shall not participate in overload resolution unless: * `CONTAINS_PROPERTY(execution::twoway_t, SupportableProperties)`, * and `CONTAINS_PROPERTY(execution::single_t, SupportableProperties)`.

Effects: Performs `e.twoway_execute(f2)`, where:

- `e` is the target object of *this;
- `f1` is the result of `DECAY_COPY(std::forward<Function>(f))`;
- `f2` is a function object of unspecified type that, when invoked as `f2()`, performs `f1()`.

Returns: A future, whose shared state is made ready when the future returned by `e.twoway_execute(f2)` is made ready, containing the result of `f1()` or any exception thrown by `f1()`. [Note: `e2.twoway_execute(f2)` may return any future type that satisfies the Future requirements, and not necessarily `std::experimental::future`. One possible implementation approach is for the polymorphic wrapper to attach a continuation to the inner future via that object's `then()` member function. When invoked, this continuation stores the result in the outer future's associated shared and makes that shared state ready. --end note]

1.4.1.3 then_t customization points

In addition to conforming to the above specification, the `then_t` property provides the following customization:

```
struct then_t
{
    template<class Executor>
        friend see-below require(Executor ex, then_t);
};
```

This customization point returns an executor that satisfies the `then_t` requirements by adapting the native functionality of an executor that does not satisfy the `then_t` requirements.

```
template<class Executor>
    friend see-below require(Executor ex, then_t);
```

Returns: A value `e1` of type `E1` that holds a copy of `ex`. `E1` has member functions `require` and `query` that forward to the corresponding members of the copy of `ex`, if present. `e1` has the same properties as `ex`, except for the addition of the `then_t` property and the exclusion of other interface-changing properties. The type `E1` satisfies the `ThenExecutor` requirements by implementing member function `then_execute` in terms of the member function `bulk_then_execute` of the object `ex`, and `E1` has a member function `Executor require(bulk_then_t) const` that returns a copy of `ex`.

Remarks: This function shall not participate in overload resolution unless `then_t::static_query_v<Executor>` is false and `bulk_then_t::static_query_v<Executor>` is true.

1.4.1.4 then_t polymorphic wrapper

TODO

1.4.1.5 bulk_twoway_t customization points

In addition to conforming to the specification for polymorphic executor wrappers, the nested class template `bulk_twoway_t::polymorphic_executor_type` provides the following member functions:

```
struct bulk_twoway_t
{
    template<class Executor>
        friend see-below require(Executor ex, bulk_twoway_t);
};
```

This customization point returns an executor that satisfies the `bulk_twoway_t` requirements by adapting the native functionality of an executor that does not satisfy the `bulk_twoway_t` requirements.

```
template<class Executor>
    friend see-below require(Executor ex, bulk_twoway_t);
```

Returns: A value `e1` of type `E1` that holds a copy of `ex`. `E1` has member functions `require` and `query` that forward to the corresponding members of the copy of `ex`, if present. For some type `T`, the type yielded by `executor_future_t<E1, T>` is `executor_future_t<Executor, T>` if `bulk_then_t::static_query_v<Executor>` is true; otherwise, it is `std::experimental::future<T>`. `e1` has the same properties as `ex`, except for the addition of the `bulk_twoway_t` property and the exclusion of other interface-changing properties. The type `E1` satisfies the `BulkTwowayExecutor` requirements as follows:

- If `twoway_t::static_query_v<Executor>` is true, then `E1` implements member function `bulk_twoway_execute` in terms of the member function `twoway_execute` of the object `ex`, and `E1` has a member function `Executor require(twoway_t) const` that returns a copy of `ex`.
- If `bulk_then_t::static_query_v<Executor>` is true, then `E1` implements member function `bulk_twoway_execute` in terms of the member function `bulk_then_execute` of the object `ex`, and `E1` has a member function `Executor require(bulk_then_t) const` that returns a copy of `ex`.
- If `then_t::static_query_v<Executor>` is true, then `E1` implements member function `bulk_twoway_execute` in terms of the member function `then_execute` of the object `ex`, and `E1` has a member function `Executor require(then_t) const` that returns a copy of `ex`.
- If `std::execution::bulk_oneway_t::static_query_v<Executor>` && `adaptable_blocking_t::static_query_v<Executor>` is true, then `E1` implements member function `bulk_twoway_execute` in terms of the member function `bulk_execute` of the object `ex`, and `E1` has a member function `Executor require(std::execution::bulk_oneway_t) const` that returns a copy of `ex`.
- If `std::execution::oneway_t::static_query_v<Executor>` && `adaptable_blocking_t::static_query_v<Executor>` is true, then `E1` implements member function `bulk_twoway_execute` in terms of the member function `execute` of the object `ex`, and `E1` has a member function `Executor require(std::execution::oneway_t) const` that returns a copy of `ex`.

Remarks: This function shall not participate in overload resolution unless `bulk_twoway_t::template static_query_v<Executor>` is false and `twoway_t::static_query_v<Executor>` || `bulk_then_t::static_query_v<Executor>` || `then_t::static_query_v<Executor>` || `(std::execution::bulk_oneway_t::static_query_v<Executor> && adaptable_blocking_t::static_query_v<Executor>)` || `(std::execution::oneway_t::static_query_v<Executor> && adaptable_blocking_t::static_query_v<Executor>)` is true.

1.4.1.6 bulk_twoway_t polymorphic wrapper

In addition to conforming to the above specification, the nested class template `bulk_twoway_t::polymorphic_executor_type` has the following member function to satisfy the BulkTowWayExecutor requirements.

```
template <class... SupportableProperties>
class polymorphic_executor_type
{
public:
    template<class Executor>
        polymorphic_executor_type(Executor e);

    template<class Executor>
        polymorphic_executor_type& operator=(Executor e);

    template<class Function, class ResultFactory, class SharedFactory>
        std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
            bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf) const;
};
```

`bulk_twoway_t::polymorphic_executor_type` satisfies the BulkTowWayExecutor requirements.

```
template<class Executor>
    polymorphic_executor_type(Executor e);
```

Remarks: This function shall not participate in overload resolution unless:

- `can_require_v<Executor, bulk_twoway_t>`.
- `can_require_v<Executor, P>`, if `P::is_requirable`, where `P` is each property in `SupportableProperties...`.
- `can_prefer_v<Executor, P>`, if `P::is_preferable`, where `P` is each property in `SupportableProperties...`.
- and `can_query_v<Executor, P>`, if `P::is_requirable == false` and `P::is_preferable == false`, where `P` is each property in `SupportableProperties...`.

Effects: *this targets a copy of `e1`, where `e1` is the result of `execution::require(e, bulk_twoway)`.

```
template<class Executor>
    polymorphic_executor_type& operator=(Executor e);
```

Requires: As for `template<class Executor> polymorphic_executor_type(Executor e)`.

Effects: `polymorphic_executor_type(std::move(e)).swap(*this)`.

Returns: *this.

```
template<class Function, class ResultFactory, class SharedFactory>
    std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
        void bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf) const;
```

Remarks: This function shall not participate in overload resolution unless: * `CONTAINS_PROPERTY(execution::twoway_t, SupportableProperties)`, * and `CONTAINS_PROPERTY(execution::bulk_t, SupportableProperties)`.

Effects: Performs `e.bulk_twoway_execute(f2, n, rf2, sf2)`, where:

- `e` is the target object of *this;
- `rf1` is the result of `DECAY_COPY(std::forward<ResultFactory>(rf))`;
- `rf2` is a function object of unspecified type that, when invoked as `rf2()`, performs `rf1()`;
- `sf1` is the result of `DECAY_COPY(std::forward<SharedFactory>(rf))`;
- `sf2` is a function object of unspecified type that, when invoked as `sf2()`, performs `sf1()`;
- if `decltype(rf1())` is non-void and `decltype(rf2())` is void, `r1` is the result of `rf1()`;
- if `decltype(rf2())` is non-void, `r2` is the result of `rf2()`;
- `s1` is the result of `sf1()`;
- `s2` is the result of `sf2()`;
- `f1` is the result of `DECAY_COPY(std::forward<Function>(f))`;
- if `decltype(rf1())` is non-void and `decltype(rf2())` is non-void, `f2` is a function object of unspecified type that, when invoked as `f2(i, r2, s2)`, performs `f1(i, r1, s1)`, where `i` is a value of type `size_t`.
- if `decltype(rf1())` is non-void and `decltype(rf2())` is void, `f2` is a function object of unspecified type that, when invoked as `f2(i, s2)`, performs `f1(i, r1, s1)`, where `i` is a value of type `size_t`.
- if `decltype(rf1())` is void and `decltype(rf2())` is non-void, `f2` is a function object of unspecified type that, when invoked as `f2(i, r2, s2)`, performs `f1(i, s1)`, where `i` is a value of type `size_t`.
- if `decltype(rf1())` is void and `decltype(rf2())` is void, `f2` is a function object of unspecified type that, when invoked as `f2(i, s2)`, performs `f1(i, s1)`, where `i` is a value of type `size_t`.

Returns: A future, whose shared state is made ready when the future returned by `e.bulk_twoway_execute(f2, n, rf2, sf2)` is made ready, containing the result in `r1` (if `decltype(rf1())` is non-void) or any exception thrown by an invocation `f1`. [Note: `e.bulk_twoway_execute(f2)` may return any future type that satisfies the Future requirements, and not necessarily `std::experimental::future`. One possible implementation approach is for the polymorphic wrapper to attach a continuation to the inner future via that object's `then()` member function. When invoked, this continuation stores the result in the outer future's associated shared and makes that shared state ready. --end note]

1.4.1.7 bulk_then_t customization points

In addition to conforming to the above specification, the bulk_then_t property provides the following customization:

```
struct bulk_then_t
{
    template<class Executor>
        friend see-below require(Executor ex, bulk_then_t);
};
```

This customization point returns an executor that satisfies the bulk_then_t requirements by adapting the native functionality of an executor that does not satisfy the bulk_then_t requirements.

```
template<class Executor>
    friend see-below require(Executor ex, bulk_then_t);
```

Returns: A value e1 of type E1 that holds a copy of ex. E1 has member functions require and query that forward to the corresponding members of the copy of ex, if present. e1 has the same properties as ex, except for the addition of the bulk_then_t property and the exclusion of other interface-changing properties. The type E1 satisfies the ThenExecutor requirements by implementing member function bulk_then_execute in terms of the member function then_execute of the object ex, and E1 has a member function Executor require(then_t) const that returns a copy of ex.

Remarks: This function shall not participate in overload resolution unless bulk_then_t::static_query_v<Executor> is false and then_t::static_query_v<Executor> is true.

1.4.1.8 bulk_then_t polymorphic wrapper

TODO

1.5 Executor type traits

1.5.1 Determining that a type satisfies executor type requirements

```
template<class T> struct is_twoway_executor;
template<class T> struct is_then_executor;
template<class T> struct is_bulk_twoway_executor;
template<class T> struct is_bulk_then_executor;
```

This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a UnaryTypeTrait (C++Std [meta.rqmts]) with a BaseCharacteristic of true_type if the corresponding condition is true, otherwise false_type.

Template	Condition	Preconditions
template<class T> struct is_twoway_executor	T meets the syntactic requirements for TwoWayExecutor.	T is a complete type.
template<class T> struct is_then_executor	T meets the syntactic requirements for ThenExecutor.	T is a complete type.
template<class T> struct is_bulk_twoway_executor	T meets the syntactic requirements for BulkTwoWayExecutor.	T is a complete type.
template<class T> struct is_bulk_then_executor	T meets the syntactic requirements for BulkThenExecutor.	T is a complete type.

1.5.2 Associated future type

```
template<class Executor, class T>
struct executor_future
{
    using type = decltype(execution::require(declval<const Executor&>(), execution::twoway).twoway_execute(declval<T(*)>()>()));
};
```